



UNIVERSIDAD
NACIONAL
DE COLOMBIA

Segunda Entrega: Spotify Churn Predictor

Natalia Ospina Agudelo

Jhon Alexander Gonzales

Valentina Espejo Peñaloza

Facultad de ingeniería y arquitectura

Universidad Nacional de Colombia

Ciencia de los datos

José Albeiro Montes Gil

2025

Descripción breve del proyecto

Spotify Churn Predictor es una aplicación web que permite predecir si un usuario abandonará o continuará utilizando Spotify, utilizando modelos de Machine Learning previamente entrenados.

El sistema funciona en dos partes:

- Frontend (la interfaz gráfica)
Desarrollado con HTML, CSS y JavaScript, permite que el usuario ingrese datos como edad, genero, país, tipo de suscripción, tiempo de escucha, tasa de salto, canciones por día, tipo de dispositivo, anuncios por semana, modo sin conexión. Algunos de estos datos son de tipo numérico y otros categóricos, para estos últimos la interfaz muestra una serie de opciones para que el usuario elija
- Backend (el servidor)
Desarrollado en Python con Flask, recibe los datos enviados desde el frontend, usa modelos de Machine Learning (.pkl) para predecir el comportamiento del usuario y devuelve los resultados.

La aplicación incluye tres modelos distintos:

- KNN (K-Nearest Neighbors): Modelo de clasificación basado en distancias. Su principio fundamental es: "Un usuario pertenece a la clase mayoritaria entre sus vecinos más cercanos."
En el desarrollo de la aplicación web el parámetro que se estableció para realizar la predicción fue `n_neighbors = 21`, lo que indica que el modelo analiza los 21 vecinos más cercanos.
- MLP (Multi-Layer Perceptron): Es una red neuronal artificial con varias capas ocultas. Aprende patrones complejos y no lineales.
En este modelo se usó `hidden_layer_sizes = (30, 15)`, esto hace referencia a que la red tiene una primera capa oculta con 30 neuronas y una segunda capa oculta con 15 neuronas.
Adicional a esto se utilizó una función de activación sigmoide, ideal para problemas de clasificación binaria como el abandono y se estableció un máximo de 500 iteraciones
- Regresión Lineal: Es un modelo estadístico ampliamente usado para clasificación binaria. Devuelve el resultado posible de abandono teniendo en cuenta valores entre 0 y 1.

Programas obligatorios

Para poder ejecutar y desplegar correctamente la aplicación Spotify Churn Predictor, es necesario instalar algunos programas fundamentales.

1. Python

Python es un lenguaje de programación ampliamente utilizado en aplicaciones web, análisis de datos, machine learning e inteligencia artificial. Este es necesario debido a que el backend de esta aplicación está construido con Flask, una librería escrita en Python.

Sin Python, el backend no podría ejecutarse.

La descarga de este se realiza mediante el siguiente link:

<https://www.python.org/downloads/>,

Durante la instalación se debe marcar la opción: Add Python to PATH, la cual permitirá usar Python desde la terminal.

2. Pip (gestor de paquetes de Python)

pip es un programa que se instala junto con Python, este permite descargar e instalar librerías adicionales, en el desarrollo del proyecto se hizo uso de varias librerías como: Flask, pandas, scikit-learn, joblib, CORS; que no vienen incluidas con Python.

Dependencias instaladas con pip

El archivo requirements.txt contiene la lista de librerías que el backend debe instalar para funcionar.

Se instala cada una desde la terminal escribiendo “pip install” y el nombre de la librería, a continuación, se explica cada una de ellas:

Flask

Flask es un framework ligero de Python para crear servidores web. En el proyecto se hace uso para recibir los datos enviados desde el frontend, prepararlos para el modelo, ejecutar los modelos de Machine Learning y devolver el resultado al navegador.

CORS

Es una extensión de Flask que habilita CORS (Cross-Origin Resource Sharing).

Cuando el HTML se abre desde el computador, y el backend corre en `http://127.0.0.1:5000`, el navegador puede bloquear la comunicación por motivos de seguridad.

Flask-CORS evita ese bloqueo.

Por eso en `app.py` aparece: `CORS(app)`

Scikit-learn

Es una librería de Python usada para Machine Learning. Esta se encargó de:

- entrenar los modelos KNN, MLP y Regresión,
- permitir cargarlos desde .pkl,

- ejecutar las predicciones.

Sin scikit-learn, los .pkl no funcionarán.

Joblib

Es Una librería usada para guardar y cargar modelos entrenados en archivos .pkl.

En el desarrollo del proyecto los modelos fueron guardados así: "knn.pkl", "mlp.pkl", "regresion.pkl". Y gracias a esta librería en el backend se cargan así:

```
knn = joblib.load('backend/knn.pkl')
```

```
mlp = joblib.load('backend/mlp.pkl')
```

```
regresion = joblib.load('backend/regresion.pkl')
```

Sin joblib, no se podrian cargar los modelos entrenados.

Pandas

Es una librería de Python que permite manejar datos en forma de tablas, conocidas como DataFrames

Numpy

Es la librería base para cálculos numéricos en Python. scikit-learn y pandas dependen de numpy para trabajar internamente.

Navegador web

En este paso se pueden usar programas como Google Chrome, Firefox, Microsoft Edge, etc.

Este navegador es necesario debido a que es el encargado de mostrar la interfaz web del proyecto la cual está hecha con:

- HTML (estructura)
- CSS (estilos)
- JavaScript (interactividad)

Adicionalmente permite al usuario ingresar los datos y mostrar la predicción recibida desde el backend.

Para abrir la aplicación web en el navegador se debe abrir el archivo: index.html

Backend

El backend se encarga de recibir los datos del usuario, convertirlos en un DataFrame, ejecutar la predicción con cada modelo y enviar los resultados al frontend.

El flujo de los datos acá es de la siguiente manera:

1. El usuario envía información mediante un formulario.
2. El servidor Flask recibe esta información en formato JSON.
3. Los datos se organizan en un DataFrame de pandas.
4. Se cargan los modelos .pkl usando joblib.
5. Cada modelo genera su predicción.
6. El servidor devuelve los resultados al navegador.

Frontend

El frontend es responsable de la interfaz que usa el público. Está compuesto por:

- HTML para la estructura visual.
- CSS para estilos.
- JavaScript para enviar datos al backend y mostrar resultados.

El flujo de los datos acá es de la siguiente manera:

1. El usuario llena un formulario.
2. JavaScript captura los valores.
3. Se empaquetan en un objeto JSON.
4. Se envían al backend mediante fetch().
5. Se reciben las predicciones.
6. Se actualiza la pantalla mostrando los resultados.

Como desplegar la aplicación

1. Verificar que Python esté instalado

Escribir en la terminal:

```
python --version o py --version
```

Si muestra un número como 3.10 o 3.12, está correctamente instalado.

2. Instalar las librerías del backend

Como se explicó anteriormente se puede realizar la instalación de una por una escribiendo “pip install” y el nombre de la librería o se pueden descargar todas directamente desde la carpeta del cd backend con: “pip install -r requirements.txt”

3. Ejecutar el backend

Dentro de la carpeta del backend, se debe correr: `python app.py`

Si todo está correcto, se verá lo siguiente:

* Running on <http://127.0.0.1:5000/predict>

Eso significa que el servidor está **listo para recibir solicitudes**.

4. Abrir el frontend

Ahora en la carpeta del frontend se debe abrir el archivo:

`index.html`

Con doble clic, o arrastrándolo al navegador.

5. Conectar frontend ↔ backend

Cuando el usuario llene el formulario, JavaScript enviará los datos al backend mediante:

```
fetch("http://127.0.0.1:5000/predict", { ... })
```

El backend procesará los datos y responderá así:

```
{
```

```
  "knn": "El modelo KNN (N-Nearest Neighbors) compara el usuario actual con los mas similares para predecir su comportamiento.
```

```
  Precision: 75%
```

```
  (Abandona/ No abandona)",
```

```
  "mlp": "El modelo MLP (Multi-Layer Perceptron) es una red neuronal que aprende patrones complejos de comportamiento.
```

```
  Precision: 75%
```

```
  (Abandona/ No abandona)",
```

"regresion": "La regresión lineal evalúa la relación directa entre las variables del usuario y la probabilidad de abandono.

Precision: 75%

(Abandona/ No abandona)",

}

El frontend mostrará esta información al usuario.